

INDENG 164: Project

Instructor: Ying Cui

Early due on: Friday, May 08, 2026, 11:59pm

Final due on: Friday, May 15, 2026, 11:59pm

Please upload your project report to Gradescope

- You can work alone or form a group of at most **two** students. Please make a group submission on Gradescope if you work in pairs.
- Students who submit their mini-project by the early deadline (05/08) receives a bonus of 2 points contributed to their final course grade. You are not penalized for meeting the final due date (05/15). No extension beyond this deadline is allowed.

Background: You have likely used large language models (LLMs) in your daily life to assist with studying and decision-making. Regardless of which LLM-based chatbot you use (e.g., ChatGPT, Gemini, or Claude), you may have noticed that these systems allow you to switch among different model modes (e.g., *Fast* versus *Thinking*).

Have you also tried AI-assisted integrated development environments (IDEs) such as Cursor? When requesting help from a coding agent, you may either manually select a specific model from a long list ¹, or allow Cursor to automatically choose a model for you. Have you experienced difficulty deciding which model to use, or wondered how Cursor selects the most suitable model for your task?

To formalize these questions, *LLM routing* has emerged as an important topic in the AI research community. Given a pool of user prompts $p \in \mathcal{P}$ and a finite set of models $m \in \mathcal{M}$, LLM routing in its basic form can be viewed as a *classification* ² problem. Specifically, we aim to learn a function $f : \mathcal{P} \rightarrow \mathcal{M}$ such that, for a given prompt $p \in \mathcal{P}$, the system selects an *optimal* model:

$$m^* := f(p).$$

The core design question is how to define an appropriate notion of optimality for the function f . A fundamental consideration is the trade-off between cost and performance. For example, using a lightweight model (e.g., Gemini 2.5 Flash) may incur very low cost (e.g., \$0.3/2.5 per million input/output tokens) even for complex prompts such as generating a project report, but it may produce lower-quality responses. In contrast, a more advanced model (e.g., Claude 4.6 Opus Fast Mode) may yield higher-quality outputs but at significantly higher cost (e.g., \$30/150 per million input/output tokens).

In general, higher-cost models tend to produce better responses, leading to an inherent trade-off between minimizing cost and maximizing performance. If we denote the average cost and quality of

¹See <https://cursor.com/docs/models-and-pricing> for the list of models supported by Cursor.

²<https://www.ibm.com/think/topics/classification-machine-learning>

applying model m to prompt p by $C(p, m)$ and $Q(p, m)$ respectively, a standard objective in the literature is to select models via a weighted combination of these two quantities:

$$f(p) := \operatorname{argmin}_{m \in \mathcal{M}} C(p, m) - \alpha \cdot Q(p, m),$$

where α is a user-specified hyperparameter that balances the trade-off.

In the starter code, we provide a simple baseline model based on this objective. To achieve better interpretability of the trade-off and to incorporate more realistic operational constraints, you are expected to develop a constrained optimization model in this project, such as a budget-constrained or performance-constrained formulation. Please refer to the following sections for further details.

Objective: Suppose you are hired by an agentic AI company (such as Cursor) as an operations research (OR) scientist. The company has collected data (see below) that contains performance and cost information for both flagship models (e.g., GPT or Gemini) and lightweight models (e.g., DeepSeek-Qwen-7B) over a representative set of prompts spanning multiple domains.

Your task is to develop an optimization model that *curates* a small but effective pool of models from a large candidate set and defines a routing policy for assigning LLMs to incoming prompts. Your model must incorporate stochastic elements and multi-stage decisions. It should also address key research questions (see below) and provide practical recommendations for deployment.

Data³: The dataset contains cost and performance scores for 33 models evaluated on 240 prompts sampled from the following four benchmark datasets:

- (Math) AIME
- (Code) LCB
- (Knowledge) GPQA, MMLU-Pro

You will find example usage of the dataset in the starter code. Note that some models have zero cost, corresponding to open-source models that can be deployed locally without API charges.

Research or business questions: Your report should propose two questions and answer them using your optimization formulations. You may select from the following, and/or propose your own within scope:

- How large must the model pool be to reliably achieve a prescribed optimality objective? Do we observe diminishing marginal returns as the pool size increases?
- What models are selected in the optimal pool? Is the pool sufficiently diverse? Which model families (e.g., Claude, Gemini, Qwen3) offer better trade-offs? Which smaller models are most valuable? Does the routing policy distribute usage evenly across models, or is it concentrated? Do conclusions differ across benchmarks?
- How sensitive is your solution to changes in the prompt distribution? Is it robust under different ambiguity sets of response distributions?
- By comparing different optimization formulations and baseline policies, how can you justify that your proposed solution is suitable for production deployment?

³Data source: Li et al. [2026]

Modeling components to incorporate: The following list is non-exhaustive. You are encouraged to incorporate as many realistic elements as possible to build a comprehensive and meaningful model.

For components requiring additional data, you may collect external data or make reasonable assumptions. For example, if you introduce a budget constraint, you should define a plausible budget level and analyze how solutions vary across a range of values.

- **Multi-Stage Decision Variables:**

- Select a subset of models from the full candidate set;
- Assign a model to each prompt request, or design a stochastic routing policy where each model is selected with a certain probability;
- (Optional) *Cascading routing policy*: sequentially apply multiple models if earlier ones fail. For example, assume access to a zero-cost verifier that checks correctness. If the initial model fails, escalate to stronger models at additional cost. How would you model such sequential decisions?
- *Auxiliary variables*: introduce additional variables for linearization or feasibility. For instance, if quality constraints cannot always be satisfied, introduce slack variables and penalize them in the objective.

- **Constraints and Objective:**

- *Randomness and Uncertainty*: Treat prompts as random variables, where the dataset represents samples from an underlying distribution. The empirical distribution may differ from the true distribution, so it is important to study robustness under reweighted sample average approximation (SAA). For example, a coding-focused company may prioritize LCB prompts, while a tutoring system may emphasize AIME.

Additionally, model outputs are inherently stochastic, which affects both cost and performance. You may incorporate this uncertainty through assumptions or estimates derived from the data.

If you adopt a stochastic routing policy, this introduces an additional layer of *endogenous uncertainty*.

- *Objective Components and Penalties*: Consider different possible formulations to balance cost and performance:
 - * Maximize performance subject to a budget constraint;
 - * Minimize cost subject to quality constraints;
 - * Penalize undesirable outcomes, such as:
 - Violations of quality thresholds;
 - Budget overruns (e.g., quadratic penalty: $(\max\{0, \frac{1}{|\mathcal{P}|} \sum_p C(p, m^*) - \$10\})^2$).

Different formulations will yield different optimal policies depending on parameters (e.g., α , budget levels, quality thresholds). You should analyze solutions across a meaningful range of these parameters.

- *Additional realistic constraints*: To make your model more practically relevant, you should incorporate at least two additional constraints motivated by real-world deployment considerations. Below are some examples with brief context:
 - * *Model-related constraints*:

- **Limited model pool size:** In practice, companies often restrict the number of models they maintain due to engineering overhead (e.g., maintenance, monitoring, integration). You may impose a constraint on the total number of models selected.
 - **Storage or deployment constraints:** Lightweight open-source models can be deployed locally (e.g., via vLLM ⁴), but they require significant storage (from tens to hundreds of GB). A resource-constrained team may impose a total storage budget across selected models.
 - **Provider or contract constraints:** Companies may have agreements with specific providers (e.g., OpenAI or Google), which could require including certain models or excluding competitors. These can be modeled as logical constraints on the model pool.
- * *Prompt-related constraints:*
- **Coupling across prompts:** Rather than treating prompts independently, you may impose system-level requirements. For example, a coding-focused platform may require that the probability of successfully solving a random coding prompt exceeds a target threshold, which introduces coupling across routing decisions.
 - **Fairness or minimum quality guarantees:** You may require that all prompts achieve at least a certain performance level. Since this may not always be feasible, you can introduce slack variables and penalize violations.
 - **Worst-case performance:** Alternatively, instead of average performance, you may optimize for the worst-performing prompt (max-min formulation) to ensure robustness.

You are encouraged to explore additional modeling aspects beyond those listed above.

Deliverables (20 Points)

Please submit a 4–5 page comprehensive project report that includes the following components:

- **Introduction (3 points):** In your own language, introduce your AI company (especially if your model has component tuned for a particular company) and the central problem concerned in LLM routing and define your business/research questions to be explored by your optimization models. Justify why a (two/multi-stage) stochastic optimization modeling well suits the context here.
- **Optimization Model (10 points):** Provide a detailed and mathematically rigorous description of decision variables, objective function, and constraints.

Clearly define every mathematical notation with its range and an explanation, such as:

- (parameter) $\mathcal{M} := \{m\}_{m=1}^M$: the set of candidate models with indices up to M .
- (decision variable) $x_{pm} \in \{0, 1\}$, $p \in \mathcal{P}$, $m \in \mathcal{M}$: binary decision whether to assign a model m to resolve prompt p .

Formulate a stochastic/robust/chance-constrained program. Present your model in its population form, and your efforts to approximate it into a solvable sample-average form.

If linearization or convexification steps are needed, clearly derive the reduction steps.

⁴<https://github.com/vllm-project/vllm>

- **Pyomo Code (2 points):** We recommend using the provided starter code. Attach your Pyomo (or other equivalent softwares) implementation used to solve your model. Present optimal solutions in the code and report.
- **Presentation and Visualization (2 points):** Present your optimal model clearly (e.g., over the cost-performance plane). If your optimal solution depends on certain hyperparameters, you will want to plot a line representing the optimal frontier of your class of optimal policy (see the starter code for examples). If you perform any ablation study of modeling component or hyperparameter tuning with respect to parameters, show what you discover through the process. Present any additional results you find help in justifying your conclusions.
- **Discussion and Recommendations (2 points):** Reflect on the results:
 - How does your model answer the research questions?
 - What business recommendations do you want to convey to your manager?
 - What further improvements or extensions would you wish to attempt if you have more time?
- **Personal Takeaway (1 point):** Share with us a bit what you learned through the experience and whether you find the project fun to work on, or any feedback you wish to share with us.

Evaluation Criteria

Projects will be assessed according to the following:

- **Model Complexity and Innovation:** Sufficient incorporation of uncertainty and mixed (multi-stage) decisions, and realistic constraints. A simple deterministic model will earn at most 15 points.
- **Completeness and clarity:** All deliverable components are addressed thoroughly. Clear and organized presentation of your approach, findings, and insights.
- **Soundness:** Correct and coherent mathematical formulation.
- **Creativity and Critical Thinking:** Demonstrates original thinking or extension beyond the stated elements.

References

Hao Li, Yiqun Zhang, Zhaoyan Guo, Chenxu Wang, Shengji Tang, Qiaosheng Zhang, Yang Chen, Biqing Qi, Peng Ye, Lei Bai, et al. Llmrouterbench: A massive benchmark and unified framework for llm routing. *arXiv preprint arXiv:2601.07206*, 2026.